



The Coq Proof Assistant

Library Coq.Classes.Morphisms

Typeclass-based morphism definition and standard, minimal instances

Author: Matthieu Sozeau Institution: LRI, CNRS UMR 8623 -
University Paris Sud

```
Require Import Coq.Program.Basics.
Require Import Coq.Program.Tactics.
Require Import Coq.Relations.Relation_Definitions.
Require Export Coq.Classes.RelationClasses.
```

Morphisms.

We now turn to the definition of `Proper` and declare standard instances. These will be used by the `setoid_rewrite` tactic later.

A morphism for a relation `R` is a proper element of the relation. The relation `R` will be instantiated by `respectful` and `A` by an arrow type for usual morphisms.

```
Section Proper.
  Let U := Type.
  Context {A B : U}.
```

```
Class Proper (R : relation A) (m : A) : Prop :=
  proper_prf : R m m.
```

Every element in the carrier of a reflexive relation is a morphism for this relation. We use a proxy class for this case which is used internally to discharge reflexivity constraints. The `Reflexive` instance will almost always be used, but it won't apply in general to any kind of `Proper (A -> B) __` goal, making proof-search much slower. A cleaner solution would be to be able to set different priorities in different hint bases and select a particular hint database for resolution of a type class constraint.

```
Class ProperProxy (R : relation A) (m : A) : Prop :=
  proper_proxy : R m m.
```

Navigation

- Standard Library
 - Table of contents
 - Index

```
Lemma eq_proper_proxy (x : A) : ProperProxy (@eq A)
x.
```

```
Lemma reflexive_proper_proxy `(Reflexive A R) (x :
A) : ProperProxy R x.
```

```
Lemma proper_proper_proxy x `(Proper R x) :
ProperProxy R x.
```

Respectful morphisms.

The fully dependent version, not used yet.

```
Definition respectful_hetero
(A B : Type)
(C : A -> Type) (D : B -> Type)
(R : A -> B -> Prop)
(R' : forall (x : A) (y : B), C x -> D y -> Prop) :
(forall x : A, C x) -> (forall x : B, D x) -> Prop
:=
fun f g => forall x y, R x y -> R' x y (f x) (g
y).
```

The non-dependent version is an instance where we forget dependencies.

```
Definition respectful (R : relation A) (R' :
relation B) : relation (A -> B) :=
Eval compute in @respectful_hetero A A (fun _ =>
B) (fun _ => B) R (fun _ _ => R').
```

End Proper.

We favor the use of Leibniz equality or a declared reflexive relation when resolving `ProperProxy`, otherwise, if the relation is given (not an evar), we fall back to `Proper`.

```
Hint Extern 1 (ProperProxy _ _) =>
class_apply @eq_proper_proxy || class_apply
@reflexive_proper_proxy : typeclass_instances.
```

```
Hint Extern 2 (ProperProxy ?R _) =>
not_evar R; class_apply @proper_proper_proxy :
typeclass_instances.
```

Notations reminiscent of the old syntax for declaring morphisms.

```
Delimit Scope signature_scope with signature.
```

```
Module ProperNotations.
```

```
Notation " R ++> R' " := (@respectful _ _
(R%signature) (R'%signature))
(right associativity, at level 55) :
signature_scope.
```

Navigation

- Standard Library
 - Table of contents
 - Index

```

    Notation " R ==> R' " := (@respectful _ _
(R%signature) (R'%signature))
    (right associativity, at level 55) :
signature_scope.

    Notation " R --> R' " := (@respectful _ _ (flip
(R%signature)) (R'%signature))
    (right associativity, at level 55) :
signature_scope.

```

End ProperNotations.

Export ProperNotations.

Local Open Scope signature_scope.

`solve_proper` try to solve the goal `Proper (?==> ... ==>?) f` by repeated introductions and setoid rewrites. It should work fine when `f` is a combination of already known morphisms and quantifiers.

```

Ltac solve_respectful t :=
  match goal with
  | |- respectful _ _ _ _ =>
    let H := fresh "H" in
    intros ? ? H; solve_respectful ltac:
(setoid_rewrite H; t)
  | _ => t; reflexivity
end.

```

```

Ltac solve_proper := unfold Proper; solve_respectful
ltac:(idtac).

```

`f_equiv` is a clone of `f_equal` that handles setoid equivalences. For example, if we know that `f` is a morphism for `E1==>E2==>E`, then the goal `E (f x y) (f x' y')` will be transformed by `f_equiv` into the subgoals `E1 x x'` and `E2 y y'`.

```

Ltac f_equiv :=
  match goal with
  | |- ?R (?f ?x) (?f' _) =>
    let T := type of x in
    let Rx := fresh "R" in
    evar (Rx : relation T);
    let H := fresh in
    assert (H : (Rx==>R)%signature f f');
    unfold Rx in *; clear Rx; [ f_equiv | apply H;
clear H; try reflexivity ]
  | |- ?R ?f ?f' =>
    solve [change (Proper R f); eauto with
typeclass_instances | reflexivity ]

```

Navigation

- Standard Library
 - Table of contents
 - Index

```
| _ => idtac
end.
```

Section Relations.

```
Let U := Type.
Context {A B : U} (P : A -> U).
```

`forall_def` reifies the dependent product as a definition.

```
Definition forall_def : Type := forall x : A, P x.
```

Dependent pointwise lifting of a relation on the range.

```
Definition forall_relation
  (sig : forall a, relation (P a)) :
relation (forall x, P x) :=
  fun f g => forall a, sig a (f a) (g a).
```

Non-dependent pointwise lifting

```
Definition pointwise_relation (R : relation B) :
relation (A -> B) :=
  fun f g => forall a, R (f a) (g a).
```

```
Lemma pointwise_pointwise (R : relation B) :
  relation_equivalence (pointwise_relation R) (@eq A
==> R).
```

Subrelations induce a morphism on the identity.

```
Global Instance subrelation_id_proper `(subrelation
A RA RA') : Proper (RA ==> RA') id.
```

The subrelation property goes through products as usual.

```
Lemma subrelation_respectful `(subl : subrelation A
RA' RA, subr : subrelation B RB RB') :
  subrelation (RA ==> RB) (RA' ==> RB').
```

And of course it is reflexive.

```
Lemma subrelation_refl R : @subrelation A R R.
```

`Proper` is itself a covariant morphism for `subrelation`. We use an unconvertible premise to avoid looping.

```
Lemma subrelation_proper `(mor : Proper A R' m)
  `(unc : Unconvertible (relation A) R R')
  `(sub : subrelation A R' R) : Proper R m.
```

```
Global Instance proper_subrelation_proper :
  Proper (subrelation ++> eq ==> impl) (@Proper A).
```

```
Global Instance pointwise_subrelation `(sub :
```

Navigation

- Standard Library
 - Table of contents
 - Index

```

subrelation B R R') :
  subrelation (pointwise_relation R)
(pointwise_relation R') | 4.

```

For dependent function types.

```

Lemma forall_subrelation (R S : forall x : A,
relation (P x)) :
  (forall a, subrelation (R a) (S a)) -> subrelation
(forall_relation R) (forall_relation S).
End Relations.

```

```

Typeclasses Opaque respectful pointwise_relation
forall_relation.

```

```

Hint Unfold Reflexive : core.
Hint Unfold Symmetric : core.
Hint Unfold Transitive : core.

```

Resolution with subrelation: favor decomposing products over applying reflexivity for unconstrained goals.

```

Ltac subrelation_tac T U :=
  (is_ground T ; is_ground U ; class_apply
@subrelation_refl) ||
  class_apply @subrelation_respectful || class_apply
@subrelation_refl.

```

```

Hint Extern 3 (@subrelation _ ?T ?U) =>
subrelation_tac T U : typeclass_instances.

```

```

CoInductive apply_subrelation : Prop :=
do_subrelation.

```

```

Ltac proper_subrelation :=
  match goal with
  [ H : apply_subrelation |- _ ] => clear H ;
class_apply @subrelation_proper
end.

```

```

Hint Extern 5 (@Proper _ ?H _) => proper_subrelation :
typeclass_instances.

```

Essential subrelation instances for `iff`, `impl` and `pointwise_relation`.

```

Instance iff_impl_subrelation : subrelation iff impl |
2.

```

```

Instance iff_flip_impl_subrelation : subrelation iff
(flip impl) | 2.

```

We use an extern hint to help unification.

```

Hint Extern 4 (subrelation (@forall_relation ?A ?B ?R)

```

Navigation

- Standard Library
 - Table of contents
 - Index

```
(@forall_relation _ _ ?S)) =>
  apply (@forall_subrelation A B R S) ; intro :
typeclass_instances.
```

Section GenericInstances.

```
Let U := Type.
Context {A B C : U}.
```

We can build a PER on the Coq function space if we have PERs on the domain and codomain.

```
Program Instance respectful_per `(PER A R, PER B R')
: PER (R ==> R').
```

The complement of a relation conserves its proper elements.

```
Program Definition complement_proper
  `(mR : Proper (A -> A -> Prop) (RA ==> RA
==> iff) R) :
  Proper (RA ==> RA ==> iff) (complement R) := _.
```

The `flip` too, actually the `flip` instance is a bit more general.

```
Program Definition flip_proper
  `(mor : Proper (A -> B -> C) (RA ==> RB ==>
RC) f) :
  Proper (RB ==> RA ==> RC) (flip f) := _.
```

Every Transitive relation gives rise to a binary morphism on `impl`, contravariant in the first argument, covariant in the second.

Proper declarations for partial applications.

Navigation

- [Standard Library](#)
 - [Table of contents](#)
 - [Index](#)

Every Transitive relation induces a morphism by "pushing" an `R x y` on the left of an `R x z` proof to get an `R y z` goal.

Every Symmetric and Transitive relation gives rise to an equivariant morphism.

```
Lemma symmetric_equiv_flip `(Symmetric A R) :  
relation_equivalence R (flip R).
```

Coq functions are morphisms for Leibniz equality, applied only if really needed.

```
Global Instance reflexive_eq_dom_reflexive  
`(Reflexive B R') :  
  Reflexive (@Logic.eq A ==> R').
```

`respectful` is a morphism for relation equivalence.

```
Global Instance respectful_morphism :  
  Proper (relation_equivalence ++>  
relation_equivalence ++> relation_equivalence)  
    (@respectful A B).
```

`R` is Reflexive, hence we can build the needed proof.

```
Lemma Reflexive_partial_app_morphism `(Proper (A ->  
B) (R ==> R')) m, ProperProxy A R x) :  
  Proper R' (m x).
```

```
Lemma flip_respectful (R : relation A) (R' :  
relation B) :  
  relation_equivalence (flip (R ==> R')) (flip R ==>  
flip R').
```

Treating flip: can't make them direct instances as we need at least a `flip` present in the goal.

```
Lemma flip1 `(subrelation A R' R) : subrelation  
(flip (flip R')) R.
```

```
Lemma flip2 `(subrelation A R R') : subrelation R  
(flip (flip R')).
```

That's if and only if

```
Lemma eq_subrelation `(Reflexive A R) : subrelation  
(@eq A) R.
```

Once we have normalized, we will apply this instance to simplify the problem.

Navigation

- Standard Library
 - Table of contents
 - Index

```

Definition proper_flip_proper `(mor : Proper A R m)
: Proper (flip R) m := mor.

```

Every reflexive relation gives rise to a morphism, only for immediately solving goals without variables.

```

Lemma reflexive_proper `{Reflexive A R} (x : A) :
Proper R x.

```

```

Lemma proper_eq (x : A) : Proper (@eq A) x.

```

```

End GenericInstances.

```

```

Class PartialApplication.

```

```

CoInductive normalization_done : Prop :=
did_normalization.

```

```

Class Params {A : Type} (of : A) (arity : nat).

```

```

Ltac partial_application_tactic :=
  let rec do_partial_apps H m cont :=
    match m with
    | ?m' ?x => class_apply
@Reflexive_partial_app_morphism ;
      [(do_partial_apps H m' ltac:(idtac))|clear H]
    | _ => cont
  end
in
let rec do_partial H ar m :=
  lazy match ar with
  | 0%nat => do_partial_apps H m ltac:(fail 1)
  | S ?n' =>
    match m with
    | ?m' ?x => do_partial H n' m'
    end
  end
in
let params m sk fk :=
  (let m' := fresh in head_of_constr m' m ;
   let n := fresh in evar (n:nat) ;
   let v := eval compute in n in clear n ;
   let H := fresh in
    assert(H:Params m' v) by (subst m'; once
typeclasses eauto) ;
    let v' := eval compute in v in subst m';
    (sk H v' || fail 1))
  || fk
in
let on_morphism m cont :=
  params m ltac:(fun H n => do_partial H n m)
  ltac:(cont)

```

Navigation

- Standard Library
 - Table of contents
 - Index


```

in
match goal with
| [ _ : normalization_done |- _ ] => fail 1
| [ _ : @Params _ _ _ |- _ ] => fail 1
| [ |- @Proper ?T _ (?m ?x) ] =>
  match goal with
  | [ H : PartialApplication |- _ ] =>
    class_apply @Reflexive_partial_app_morphism;
[[clear H]
  | _ => on_morphism (m x)
  ltac:(class_apply
@Reflexive_partial_app_morphism)
end
end.

```

Bootstrap !!!

```

Instance proper_proper : Proper (relation_equivalence
==> eq ==> iff) (@Proper A).

```

```

Ltac proper_reflexive :=
  match goal with
  | [ _ : normalization_done |- _ ] => fail 1
  | _ => class_apply proper_eq || class_apply
@reflexive_proper
end.

```

```

Hint Extern 1 (subrelation (flip _) _) => class_apply
@flip1 : typeclass_instances.
Hint Extern 1 (subrelation _ (flip _)) => class_apply
@flip2 : typeclass_instances.

```

```

Hint Extern 1 (Proper _ (complement _)) => apply
@complement_proper
: typeclass_instances.
Hint Extern 1 (Proper _ (flip _)) => apply
@flip_proper
: typeclass_instances.
Hint Extern 2 (@Proper _ (flip _) _) => class_apply
@proper_flip_proper
: typeclass_instances.
Hint Extern 4 (@Proper _ _ _) =>
partial_application_tactic
: typeclass_instances.
Hint Extern 7 (@Proper _ _ _) => proper_reflexive
: typeclass_instances.

```

Special-purpose class to do normalization of signatures w.r.t. flip.

Section Normalize.

```
Context (A : Type).
```

```
Class Normalizes (m : relation A) (m' : relation A)
```

Navigation

- Standard Library
 - Table of contents
 - Index

```
: Prop :=
  normalizes : relation_equivalence m m'.
```

Current strategy: add `flip` everywhere and reduce using `subrelation` afterwards.

```
Lemma proper_normalizes_proper `(Normalizes R0 R1,
Proper A R1 m) : Proper R0 m.
```

```
Lemma flip_atom R : Normalizes R (flip (flip R)).
```

```
End Normalize.
```

```
Lemma flip_arrow {A : Type} {B : Type}
  `(NA : Normalizes A R (flip R'''), NB :
Normalizes B R' (flip R'')) :
  Normalizes (A -> B) (R ==> R') (flip (R''' ==>
R''))%signature).
```

```
Ltac normalizes :=
  match goal with
  | [ |- Normalizes _ (respectful _ _) _ ] =>
class_apply @flip_arrow
  | _ => class_apply @flip_atom
  end.
```

```
Ltac proper_normalization :=
  match goal with
  | [ _ : normalization_done |- _ ] => fail 1
  | [ _ : apply_subrelation |- @Proper _ ?R _ ] =>
    let H := fresh "H" in
    set(H:=did_normalization) ; class_apply
@proper_normalizes_proper
  end.
```

```
Hint Extern 1 (Normalizes _ _ _) => normalizes :
typeclass_instances.
```

```
Hint Extern 6 (@Proper _ _ _) => proper_normalization
: typeclass_instances.
```

When the relation on the domain is symmetric, we can flip the relation on the codomain. Same for binary functions.

```
Lemma proper_sym_flip :
  forall `(Symmetric A R1)`(Proper (A->B) (R1==>R2) f),
  Proper (R1==>flip R2) f.
```

```
Lemma proper_sym_flip_2 :
  forall `(Symmetric A R1)`(Symmetric B R2)`(Proper (A-
>B->C) (R1==>R2==>R3) f),
  Proper (R1==>R2==>flip R3) f.
```

Navigation

- Standard Library
 - Table of contents
 - Index

When the relation on the domain is symmetric, a predicate is compatible with `iff` as soon as it is compatible with `impl`. Same with a binary relation.

```
Lemma proper_sym_impl_iff : forall `(Symmetric A
R)`(Proper _ (R==>impl) f),
  Proper (R==>iff) f.
```

```
Lemma proper_sym_impl_iff_2 :
  forall `(Symmetric A R)`(Symmetric B R')`(Proper _
(R==>R'==>impl) f),
  Proper (R==>R'==>iff) f.
```

A `PartialOrder` is compatible with its underlying equivalence.

```
Instance PartialOrder_proper `(PartialOrder A eqA R) :
  Proper (eqA==>eqA==>iff) R.
```

From a `PartialOrder` to the corresponding `StrictOrder`: `lt = le /\ ~eq`. If the order is total, we could also say `gt = ~le`.

```
Lemma PartialOrder_StrictOrder `(PartialOrder A eqA R)
:
  StrictOrder (relation_conjunction R (complement
eqA)).
```

From a `StrictOrder` to the corresponding `PartialOrder`: `le = lt \/ eq`. If the order is total, we could also say `ge = ~lt`.

```
Lemma StrictOrder_PreOrder
  `(Equivalence A eqA, StrictOrder A R, Proper _
(eqA==>eqA==>iff) R) :
  PreOrder (relation_disjunction R eqA).
```

```
Hint Extern 4 (PreOrder (relation_disjunction _ _)) =>
  class_apply StrictOrder_PreOrder :
typeclass_instances.
```

```
Lemma StrictOrder_PartialOrder
  `(Equivalence A eqA, StrictOrder A R, Proper _
(eqA==>eqA==>iff) R) :
  PartialOrder eqA (relation_disjunction R eqA).
```

```
Hint Extern 4 (StrictOrder (relation_conjunction _ _))
=>
  class_apply PartialOrder_StrictOrder :
typeclass_instances.
```

```
Hint Extern 4 (PartialOrder _ (relation_disjunction _
_)) =>
  class_apply StrictOrder_PartialOrder :
typeclass_instances.
```

Navigation

- Standard Library
 - Table of contents
 - Index

Navigation

- [Standard Library](#)
 - [Table of contents](#)
 - [Index](#)